

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Título e Introdução

Objetivo da Aula

Conhecer outras estruturas de dados fundamentais em Python, especificamente **tuplas** e **dicionários**, entendendo suas características, diferenças e aplicações práticas.

O que você vai aprender:

- **Tuplas:** Definição, características e manipulação
- **Diferenças:** Tuplas vs Listas - quando usar cada uma
- **Dicionários:** Estrutura chave-valor e operações
- **Aplicações práticas:** Exemplos do mundo real

Importância do Tema

Tuplas e dicionários são estruturas de dados essenciais em Python. As tuplas oferecem imutabilidade e performance, enquanto os dicionários permitem armazenar dados de forma organizada e eficiente através de chaves.

Pré-requisitos

- Conhecimento básico de Python
- Familiaridade com listas
- Conceitos de variáveis e tipos de dados

Dica Importante

Esta aula complementa o conhecimento sobre listas (BEP-006). É importante entender as diferenças entre essas estruturas para escolher a mais adequada para cada situação.



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

🎯 Objetivos da Aula

🎯 Objetivo Geral

Conhecer outras estruturas de dados fundamentais em Python, especificamente tuplas e dicionários, desenvolvendo competências para escolher e utilizar a estrutura mais adequada para cada situação.

📋 Objetivos Específicos

Ao final desta aula, você será capaz de:

📁 Tuplas

- Compreender o conceito de tuplas
- Identificar características das tuplas
- Criar e manipular tuplas
- Usar métodos de tuplas

📁 Dicionários

- Entender estrutura chave-valor
- Criar e acessar dicionários
- Realizar operações CRUD
- Aplicar em situações práticas

Competências Desenvolvidas

Competência	Descrição	Nível
Análise	Identificar quando usar tuplas vs listas vs dicionários	Intermediário
Aplicação	Implementar soluções usando estruturas adequadas	Intermediário
Síntese	Combinar diferentes estruturas em soluções complexas	Avançado

💡 Metodologia

Aula Expositiva: Apresentação dos conceitos com demonstrações práticas
Aplicação de Exercícios: Prática guiada e individual no Google Colab

🕒 Duração

2 horas (120 minutos) - Tempo otimizado para teoria e prática



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

O que são Tuplas?

Definição

Uma **tupla** é uma coleção ordenada e **imutável** de elementos em Python. É similar a uma lista, mas com a diferença fundamental de que não pode ser modificada após sua criação.

Sintaxe Básica

```
# Criação de tuplas
tupla_vazia = ()
tupla_com_elementos = (1, 2, 3, 4, 5)
tupla_mista = ("Python", 3.9, True, [1, 2, 3])

# Também pode ser criada sem parênteses (em alguns casos)
tupla_simples = 1, 2, 3, 4, 5
```

Características das Tuplas

Característica	Descrição	Exemplo
Ordenada	Elementos têm posição fixa	(1, 2, 3) != (3, 2, 1)
Imutável	Não pode ser alterada após criação	tupla[0] = 10 # ERRO!
Indexada	Acesso por índice (0, 1, 2...)	tupla[0]

Característica	Descrição	Exemplo
Heterogênea	Pode conter diferentes tipos	(1, "texto", True)

Exemplo Prático

```
# Dados de uma pessoa
pessoa = ("João Silva", 25, "Engenheiro", "Salvador")

# Acessando elementos
nome = pessoa[0]      # "João Silva"
idade = pessoa[1]     # 25
profissao = pessoa[2] # "Engenheiro"
cidade = pessoa[3]    # "Salvador"

print(f"Nome: {nome}, Idade: {idade}")
```

Vantagens das Tuplas

- **Performance:** Mais rápidas que listas
- **Segurança:** Dados não podem ser alterados acidentalmente
- **Memória:** Consomem menos memória
- **Hashable:** Podem ser usadas como chaves em dicionários



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Características das Tuplas

Imutabilidade

Característica Principal

As tuplas são **imutáveis**, ou seja, uma vez criadas, não podem ser modificadas. Isso significa que você não pode adicionar, remover ou alterar elementos.

```
# Exemplo de imutabilidade
coordenadas = (10, 20)
print(coordenadas) # (10, 20)

# Tentativa de modificação (ERRO!)
# coordenadas[0] = 15 # TypeError: 'tuple' object does not support item assignment
# coordenadas.append(30) # AttributeError: 'tuple' object has no attribute 'append'
```



Comparação: Tupla vs Lista

Aspecto	Tupla	Lista
Sintaxe	()	[]
Mutabilidade	 Imutável	 Mutável
Performance	 Mais rápida	 Mais lenta
Memória	 Menos memória	 Mais memória

Aspecto	Tupla	Lista
Hashable	 Sim	 Não

Casos de Uso das Tuplas

Use Tuplas Para:

- Coordenadas (x, y)
- Configurações fixas
- Chaves de dicionários
- Retorno de múltiplos valores
- Dados que não devem mudar

NÃO Use Tuplas Para:

- Listas de compras
- Dados que mudam frequentemente
- Quando precisa adicionar/remover
- Coleções dinâmicas
- Quando a ordem pode mudar

Exemplo: Coordenadas Geográficas

```
# Coordenadas de cidades (dados fixos)
salvador = (-12.9714, -38.5014)
sao_paulo = (-23.5505, -46.6333)
rio_janeiro = (-22.9068, -43.1729)

# Usando como chave em dicionário
cidades = {
    salvador: "Salvador, BA",
    sao_paulo: "São Paulo, SP",
    rio_janeiro: "Rio de Janeiro, RJ"
```



```
}  
  
print(cidades[salvador]) # Salvador, BA
```



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Diferenças: Tuplas vs Listas

Comparação Detalhada

Entender as diferenças entre tuplas e listas é fundamental para escolher a estrutura de dados mais adequada para cada situação.

Análise Comparativa

Característica	Tupla	Lista	Impacto
Mutabilidade	 Imutável	 Mutável	Tuplas são mais seguras
Performance	 Mais rápida	 Mais lenta	Tuplas são mais eficientes
Memória	 Menos memória	 Mais memória	Tuplas economizam espaço
Métodos	count(), index()	append(), remove(), etc.	Listas têm mais funcionalidades
Hashable	 Sim	 Não	Tuplas podem ser chaves

Teste de Performance

```
import time
```



```
# Teste de criação
inicio = time.time()
tupla = tuple(range(1000000))
tempo_tupla = time.time() - inicio

inicio = time.time()
lista = list(range(1000000))
tempo_lista = time.time() - inicio

print(f"Tupla: {tempo_tupla:.4f}s")
print(f"Lista: {tempo_lista:.4f}s")
print(f"Tupla é {tempo_lista/tempo_tupla:.1f}x mais rápida")
```

Quando Usar Cada Uma?

Use Tuplas Quando:

- Dados não devem ser alterados
- Performance é crítica
- Precisa usar como chave
- Retornar múltiplos valores
- Configurações fixas

Use Listas Quando:

- Dados podem mudar
- Precisa adicionar/remover
- Coleção dinâmica
- Múltiplas operações
- Ordem pode mudar

Exemplo Prático: Sistema de Coordenadas

```
# Tupla: Coordenadas fixas de um ponto
ponto_a = (10, 20)
ponto_b = (30, 40)
```



```
# Lista: Pontos que podem ser adicionados
caminho = [ponto_a, ponto_b]
caminho.append((50, 60)) # Adicionando novo ponto
caminho.remove(ponto_a)  # Removendo ponto

# Tupla: Configuração fixa
config = ("localhost", 8080, True)

# Lista: Logs que crescem
logs = ["Erro 1", "Erro 2"]
logs.append("Erro 3") # Adicionando novo log
```

💡 Dica de Ouro

Regra prática: Se você não precisa modificar os dados após a criação, use tupla. Se precisa de flexibilidade para modificar, use lista.



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

+ Criação e Acesso a Tuplas

Formas de Criar Tuplas

```
# 1. Tupla vazia
tupla_vazia = ()
print(tupla_vazia) # ()

# 2. Tupla com elementos
cores = ("vermelho", "azul", "verde")
print(cores) # ('vermelho', 'azul', 'verde')

# 3. Tupla com um elemento (vírgula obrigatória!)
tupla_um_elemento = (42,) # Note a vírgula!
print(tupla_um_elemento) # (42,)

# 4. Sem parênteses (em alguns casos)
numeros = 1, 2, 3, 4, 5
print(numeros) # (1, 2, 3, 4, 5)

# 5. Usando tuple()
lista = [1, 2, 3]
tupla_da_lista = tuple(lista)
print(tupla_da_lista) # (1, 2, 3)
```



Acesso aos Elementos

```
dados = ("João", 25, "Engenheiro", "Salvador")

# Acesso por índice positivo
nome = dados[0] # "João"
idade = dados[1] # 25
profissao = dados[2] # "Engenheiro"
cidade = dados[3] # "Salvador"

# Acesso por índice negativo
ultimo = dados[-1] # "Salvador"
penultimo = dados[-2] # "Engenheiro"

print(f"Nome: {nome}, Idade: {idade}")
```



```
print(f"Último elemento: {ultimo}")
```

Fatiamento (Slicing)

```
numeros = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

# Fatiamento básico
primeiros = numeros[:3] # (0, 1, 2)
ultimos = numeros[7:] # (7, 8, 9)
meio = numeros[3:7] # (3, 4, 5, 6)

# Fatiamento com passo
pares = numeros[::2] # (0, 2, 4, 6, 8)
impares = numeros[1::2] # (1, 3, 5, 7, 9)
invertida = numeros[::-1] # (9, 8, 7, 6, 5, 4, 3, 2, 1, 0)

print(f"Primeiros 3: {primeiros}")
print(f"Pares: {pares}")
print(f"Invertida: {invertida}")
```



Desempacotamento

```
# Desempacotamento simples
coordenadas = (10, 20)
x, y = coordenadas
print(f"X: {x}, Y: {y}") # X: 10, Y: 20

# Desempacotamento com múltiplos valores
pessoa = ("Maria", 30, "Médica", "São Paulo")
nome, idade, profissao, cidade = pessoa
print(f"{nome} tem {idade} anos e é {profissao} em {cidade}")

# Desempacotamento parcial
primeiro, *meio, ultimo = (1, 2, 3, 4, 5)
print(f"Primeiro: {primeiro}") # 1
print(f"Meio: {meio}") # [2, 3, 4]
print(f"Último: {ultimo}") # 5
```



Exemplo: Sistema de Coordenadas

```
# Pontos em um plano cartesiano
ponto_a = (3, 4)
ponto_b = (6, 8)

# Desempacotando para cálculos
x1, y1 = ponto_a
```




```
x2, y2 = ponto_b

# Calculando distância
distancia = ((x2 - x1)**2 + (y2 - y1)**2)**0.5
print(f"Distância entre pontos: {distancia:.2f}")

# Criando tupla de resultado
resultado = (distancia, "unidades")
print(f"Resultado: {resultado}")
```

⚠ Cuidados Importantes

- **Tupla com um elemento:** Sempre use vírgula: (42,)
- **Índices:** Lembre-se que começam em 0
- **Fatiamento:** O último índice é exclusivo
- **Desempacotamento:** Número de variáveis deve coincidir



PROGRAMA DE
FORMAÇÃO ACELERADA



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Métodos de Tuplas



Métodos Disponíveis

As tuplas têm apenas **2 métodos** devido à sua natureza imutável: `count()` e `index()`.

1
2
3
4

Método count()

```
# Conta quantas vezes um elemento aparece
cores = ("vermelho", "azul", "verde", "azul", "amarelo", "azul")

# Contando ocorrências
qtd_azul = cores.count("azul")
qtd_verde = cores.count("verde")
qtd_roxo = cores.count("roxo")

print(f"Quantidade de 'azul': {qtd_azul}") # 3
print(f"Quantidade de 'verde': {qtd_verde}") # 1
print(f"Quantidade de 'roxo': {qtd_roxo}") # 0
```



Método index()

```
# Encontra a posição da primeira ocorrência
frutas = ("maçã", "banana", "laranja", "banana", "uva")

# Encontrando posições
pos_banana = frutas.index("banana") # 1
pos_laranja = frutas.index("laranja") # 2
pos_uva = frutas.index("uva") # 4

print(f"Posição de 'banana': {pos_banana}")
print(f"Posição de 'laranja': {pos_laranja}")
```



```
print(f"Posição de 'uva': {pos_uva}")

# Erro se elemento não existir
# pos_morango = frutas.index("morango") # ValueError
```



Operações com Tuplas

```
# Concatenação (+)
tupla1 = (1, 2, 3)
tupla2 = (4, 5, 6)
concatenada = tupla1 + tupla2
print(f"Concatenação: {concatenada}") # (1, 2, 3, 4, 5, 6)

# Repetição (*)
repetida = tupla1 * 3
print(f"Repetição: {repetida}") # (1, 2, 3, 1, 2, 3, 1, 2, 3)

# Verificação de pertinência (in)
numeros = (1, 2, 3, 4, 5)
print(f"3 está na tupla: {3 in numeros}") # True
print(f"6 está na tupla: {6 in numeros}") # False
```



Funções Built-in

```
# Funções que funcionam com tuplas
notas = (8.5, 7.0, 9.2, 6.8, 8.0)

# Informações básicas
print(f"Tamanho: {len(notas)}") # 5
print(f"Maior nota: {max(notas)}") # 9.2
print(f"Menor nota: {min(notas)}") # 6.8
print(f"Soma: {sum(notas)}") # 39.5
print(f"Média: {sum(notas)/len(notas):.2f}") # 7.90

# Verificação de tipos
print(f"É tupla: {isinstance(notas, tuple)}") # True
print(f"É lista: {isinstance(notas, list)}") # False
```



Exemplo: Sistema de Pontuação

```
# Pontuações de um jogo
pontuacoes = (100, 150, 200, 150, 300, 150, 250)

# Análise das pontuações
total_pontos = sum(pontuacoes)
pontuacao_maxima = max(pontuacoes)
```



```
pontuacao_minima = min(pontuacoes)
media_pontos = total_pontos / len(pontuacoes)

# Contando pontuações específicas
qtd_150 = pontuacoes.count(150)
pos_300 = pontuacoes.index(300)

print(f"Total de pontos: {total_pontos}")
print(f"Pontuação máxima: {pontuacao_maxima}")
print(f"Pontuação mínima: {pontuacao_minima}")
print(f"Média: {media_pontos:.2f}")
print(f"Quantas vezes 150: {qtd_150}")
print(f"Posição do 300: {pos_300}")
```

Resumo dos Métodos

Método	Função	Exemplo
<code>count(x)</code>	Conta ocorrências de x	<code>tupla.count(5)</code>
<code>index(x)</code>	Posição da primeira ocorrência de x	<code>tupla.index(5)</code>



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

O que são Dicionários?

Definição

Um **dicionário** é uma coleção de pares **chave-valor** em Python. É uma estrutura de dados que permite armazenar e acessar informações de forma organizada e eficiente.

Conceito Chave-Valor

```
# Estrutura básica: chave : valor
contato = {
    "nome": "João Silva",
    "idade": 25,
    "telefone": "(71) 99999-8888",
    "email": "joao@email.com"
}

# Acessando valores através das chaves
print(contato["nome"])    # João Silva
print(contato["idade"])   # 25
print(contato["telefone"]) # (71) 99999-8888
```



Características dos Dicionários

Característica	Descrição	Exemplo
Mutável	Pode ser modificado após criação	dicio["nova_chave"] = "valor"

Característica	Descrição	Exemplo
Indexado por chave	Acesso através de chaves, não índices	dicio["chave"]
Chaves únicas	Cada chave aparece apenas uma vez	Chaves duplicadas sobrescrevem
Heterogêneo	Chaves e valores podem ser de tipos diferentes	{1: "um", "dois": 2}

Formas de Criar Dicionários

```
# 1. Dicionário vazio
dicio_vazio = {}
dicio_vazio2 = dict()

# 2. Dicionário com elementos
pessoa = {
    "nome": "Maria",
    "idade": 30,
    "profissao": "Engenheira"
}

# 3. Usando dict() com pares
cores = dict(vermelho="#FF0000", azul="#0000FF", verde="#00FF00")

# 4. A partir de lista de tuplas
pares = [("a", 1), ("b", 2), ("c", 3)]
dicio_pares = dict(pares)

# 5. Comprehension
quadrados = {x: x**2 for x in range(1, 6)}
print(quadrados) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```



Casos de Uso dos Dicionários

Use Dicionários Para:

- Agenda telefônica
- Configurações de sistema
- Inventário de produtos

- Dados de usuários
- Mapeamento de valores
- Contadores e frequências

⊗ NÃO Use Dicionários Para:

- Listas ordenadas simples
- Quando a ordem é importante
- Coleções com elementos duplicados
- Operações matemáticas em sequência

💡 Exemplo: Sistema de Inventário

```
# Inventário de uma loja
inventario = {
    "notebook": {
        "preco": 2500.00,
        "estoque": 10,
        "categoria": "Eletrônicos"
    },
    "mouse": {
        "preco": 50.00,
        "estoque": 25,
        "categoria": "Acessórios"
    },
    "teclado": {
        "preco": 120.00,
        "estoque": 15,
        "categoria": "Acessórios"
    }
}

# Acessando informações
produto = "notebook"
info = inventario[produto]
print(f"{produto.title()}:")
print(f"  Preço: R$ {info['preco']:.2f}")
print(f"  Estoque: {info['estoque']} unidades")
print(f"  Categoria: {info['categoria']}")
```

💡 Analogia

Um dicionário é como um **dicionário de palavras**: você procura uma palavra (chave) para encontrar seu significado (valor). É uma forma muito eficiente de organizar e acessar informações.



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Estrutura de Dicionários

Anatomia de um Dicionário

```
# Estrutura: {chave1: valor1, chave2: valor2, ...}
contato = {
    "nome": "Ana Costa",          # chave: "nome", valor: "Ana Costa"
    "idade": 28,                  # chave: "idade", valor: 28
    "telefone": "(71) 99999-7777", # chave: "telefone", valor: "(71) 99999-7777"
    "email": "ana@email.com"      # chave: "email", valor: "ana@email.com"
}

# Visualizando a estrutura
print("Chaves:", contato.keys()) # dict_keys(['nome', 'idade', 'telefone', 'email'])
print("Valores:", contato.values()) # dict_values(['Ana Costa', 28, '(71) 99999-7777', 'ana@email.com'])
print("Items:", contato.items()) # dict_items([('nome', 'Ana Costa'), ('idade', 28), ('telefone', '(71) 99999-7777'), ('email', 'ana@email.com')])
```



Tipos de Chaves

```
# Chaves podem ser de diferentes tipos
dicionario_misto = {
    "string": "valor1",          # String
    42: "valor2",                # Número inteiro
    3.14: "valor3",              # Número float
    True: "valor4",              # Boolean
    (1, 2, 3): "valor5"         # Tupla (hashable)
}

# Chaves devem ser hashable (imutáveis)
# ERRO: listas não podem ser chaves
# dicio_erro = {[1, 2]: "valor"} # TypeError: unhashable type: 'list'

print(dicionario_misto["string"]) # valor1
print(dicionario_misto[42])       # valor2
print(dicionario_misto[True])     # valor4
print(dicionario_misto[(1, 2, 3)]) # valor5
```



Tipos de Valores

```
# Valores podem ser de qualquer tipo
dicionario_valores = {
    "string": "texto",
    "numero": 42,
    "float": 3.14,
    "boolean": True,
    "lista": [1, 2, 3],
    "tupla": (4, 5, 6),
    "dicionario": {"nested": "value"},
    "none": None
}

# Acessando diferentes tipos de valores
print(dicionario_valores["string"]) # texto
print(dicionario_valores["lista"])  # [1, 2, 3]
print(dicionario_valores["dicionario"]) # {'nested': 'value'}
print(dicionario_valores["dicionario"]["nested"]) # value
```



Dicionários Aninhados

```
# Dicionário dentro de dicionário
empresa = {
    "nome": "TechCorp",
    "funcionarios": {
        "joao": {
            "cargo": "Desenvolvedor",
            "salario": 5000,
            "departamento": "TI"
        },
        "maria": {
            "cargo": "Designer",
            "salario": 4000,
            "departamento": "Design"
        }
    },
    "endereco": {
        "rua": "Rua das Flores, 123",
        "cidade": "Salvador",
        "estado": "BA"
    }
}

# Acessando dados aninhados
print(f"Empresa: {empresa['nome']}")
print(f"João - Cargo: {empresa['funcionarios']['joao']['cargo']}")
print(f"Endereço: {empresa['endereco']['rua']}")
```



Tamanho e Verificações



```
dados = {"a": 1, "b": 2, "c": 3}

# Tamanho do dicionário
print(f"Tamanho: {len(dados)}") # 3

# Verificar se chave existe
print(f"'a' existe: {'a' in dados}") # True
print(f"'d' existe: {'d' in dados}") # False

# Verificar se valor existe
print(f"Valor 2 existe: {2 in dados.values()}") # True
print(f"Valor 4 existe: {4 in dados.values()}") # False

# Verificar se item existe
print(f "('a', 1) existe: {'a', 1} in dados.items()") # True
```

- **Aninhamento:** Dicionários podem conter outros dicionários



💡 Exemplo: Sistema de Notas



```
# Sistema de notas de alunos
notas_alunos = {
    "Ana": {
        "matematica": [8.5, 7.0, 9.2],
        "portugues": [7.5, 8.0, 8.5],
        "ciencias": [9.0, 8.5, 9.5]
    },
    "Bruno": {
        "matematica": [6.8, 8.0, 7.5],
        "portugues": [7.2, 6.5, 8.0],
        "ciencias": [8.0, 7.5, 8.5]
    }
}

# Calculando média de um aluno
aluno = "Ana"
materia = "matematica"
notas = notas_alunos[aluno][materia]
media = sum(notas) / len(notas)

print(f"{aluno} - {materia.title()}: {media:.2f}")
```

📌 Dicas Importantes

- **Chaves únicas:** Chaves duplicadas sobrescrevem valores anteriores
- **Hashable:** Chaves devem ser imutáveis (strings, números, tuplas)
- **Flexibilidade:** Valores podem ser de qualquer tipo

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

+ Criação de Dicionários

Métodos de Criação

```
# 1. Dicionário vazio
dicio_vazio = {}
dicio_vazio2 = dict()

# 2. Dicionário com elementos
pessoa = {
    "nome": "Carlos",
    "idade": 35,
    "profissao": "Médico"
}

# 3. Usando dict() com argumentos nomeados
cores = dict(vermelho="#FF0000", azul="#0000FF", verde="#00FF00")

# 4. A partir de lista de tuplas
pares = [("a", 1), ("b", 2), ("c", 3)]
dicio_pares = dict(pares)

# 5. A partir de duas listas (zip)
chaves = ["nome", "idade", "cidade"]
valores = ["João", 25, "Salvador"]
dicio_zip = dict(zip(chaves, valores))
print(dicio_zip) # {'nome': 'João', 'idade': 25, 'cidade': 'Salvador'}
```



Dictionary Comprehension

```
# Criando dicionário com comprehension
# Sintaxe: {chave: valor for item in iterável}

# Quadrados dos números
quadrados = {x: x**2 for x in range(1, 6)}
print(quadrados) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Com condição
pares_quadrados = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(pares_quadrados) # {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```



```
# A partir de lista
nomes = ["Ana", "Bruno", "Carla"]
iniciais = {nome: nome[0] for nome in nomes}
print(iniciais) # {'Ana': 'A', 'Bruno': 'B', 'Carla': 'C'}

# Transformando lista em dicionário
notas = [8.5, 7.0, 9.2, 6.8]
notas_dict = {f"prova_{i+1}": nota for i, nota in enumerate(notas)}
print(notas_dict) # {'prova_1': 8.5, 'prova_2': 7.0, 'prova_3': 9.2, 'prova_4': 6.8}
```

Criação Dinâmica

```
# Criando dicionário dinamicamente
contatos = {}

# Adicionando contatos um por um
contatos["João"] = {"telefone": "99999-1111", "email": "joao@email.com"}
contatos["Maria"] = {"telefone": "99999-2222", "email": "maria@email.com"}
contatos["Pedro"] = {"telefone": "99999-3333", "email": "pedro@email.com"}

print(contatos)

# Usando update() para adicionar múltiplos itens
novos_contatos = {
    "Ana": {"telefone": "99999-4444", "email": "ana@email.com"},
    "Bruno": {"telefone": "99999-5555", "email": "bruno@email.com"}
}
contatos.update(novos_contatos)
print(contatos)
```



Dicionários com Dados Estruturados

```
# Criando dicionário com dados estruturados
produtos = {
    "notebook": {
        "preco": 2500.00,
        "estoque": 10,
        "categoria": "Eletrônicos",
        "especificacoes": {
            "processador": "Intel i7",
            "memoria": "16GB",
            "armazenamento": "512GB SSD"
        }
    },
    "mouse": {
        "preco": 50.00,
        "estoque": 25,
        "categoria": "Acessórios",
        "especificacoes": {
            "tipo": "Óptico",
            "conexao": "USB",

```




```
        "dpi": 1600
    }
}

# Acessando dados aninhados
produto = "notebook"
print(f"{produto.title()}:")
print(f"    Preço: R$ {produtos[produto]['preco']:.2f}")
print(f"    Processador: {produtos[produto]['especificacoes']['processador']}")
```

- **Legibilidade:** Use estruturas claras e consistentes



💡 Exemplo: Sistema de Configuração

```
# Sistema de configuração de aplicação
config = {
    "database": {
        "host": "localhost",
        "port": 5432,
        "name": "meuapp",
        "user": "admin",
        "password": "senha123"
    },
    "server": {
        "host": "0.0.0.0",
        "port": 8080,
        "debug": True,
        "ssl": False
    },
    "logging": {
        "level": "INFO",
        "file": "app.log",
        "max_size": "10MB"
    }
}

# Acessando configurações
db_config = config["database"]
print(f"Banco: {db_config['host']}:{db_config['port']}")
print(f"Debug: {config['server']['debug']}")
```

⚠ Cuidados na Criação

- **Chaves duplicadas:** A última chave sobrescreve as anteriores
- **Chaves hashable:** Use apenas tipos imutáveis como chaves
- **Performance:** Dictionary comprehension é mais eficiente

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Acesso a Dicionários

Métodos de Acesso

```

pessoa = {
    "nome": "Ana Silva",
    "idade": 28,
    "profissao": "Designer",
    "cidade": "Salvador"
}

# 1. Acesso direto com []
nome = pessoa["nome"]
idade = pessoa["idade"]
print(f"Nome: {nome}, Idade: {idade}")

# 2. Método get() - mais seguro
nome = pessoa.get("nome")
telefone = pessoa.get("telefone") # Retorna None se não existir
telefone_default = pessoa.get("telefone", "Não informado")

print(f"Nome: {nome}")
print(f"Telefone: {telefone}") # None
print(f"Telefone: {telefone_default}") # Não informado
```



Acesso Seguro

```

# Comparação entre acesso direto e get()
dados = {"nome": "João", "idade": 25}

# Acesso direto - pode gerar erro
# profissao = dados["profissao"] # KeyError!

# Acesso seguro com get()
profissao = dados.get("profissao", "Não informado")
print(f"Profissão: {profissao}") # Não informado

# Verificando existência antes de acessar
if "profissao" in dados:
    profissao = dados["profissao"]
```



```

else:
    profissao = "Não informado"
print(f"Profissão: {profissao}")
```

Iterando sobre Dicionários

```

contato = {
    "nome": "Maria",
    "telefone": "99999-8888",
    "email": "maria@email.com",
    "cidade": "Salvador"
}

# Iterando sobre chaves
print("Chaves:")
for chave in contato.keys():
    print(f" {chave}")

# Iterando sobre valores
print("\nValores:")
for valor in contato.values():
    print(f" {valor}")

# Iterando sobre pares chave-valor
print("\nPares chave-valor:")
for chave, valor in contato.items():
    print(f" {chave}: {valor}")

# Iterando diretamente (sobre chaves)
print("\nIteração direta:")
for chave in contato:
    print(f" {chave}: {contato[chave]}")
```



Acesso a Dados Aninhados

```

# Dicionário aninhado
empresa = {
    "nome": "TechCorp",
    "funcionarios": {
        "joao": {
            "cargo": "Desenvolvedor",
            "salario": 5000,
            "departamento": "TI"
        },
        "maria": {
            "cargo": "Designer",
            "salario": 4000,
            "departamento": "Design"
        }
    }
}
```



```
# Acesso a dados aninhados
print(f"Empresa: {empresa['nome']}")
print(f"João - Cargo: {empresa['funcionarios']['joao']['cargo']}")
print(f"João - Salário: R$ {empresa['funcionarios']['joao']['salario']}")

# Acesso seguro a dados aninhados
joao_salario = empresa.get("funcionarios", {}).get("joao", {}).get("salario", 0)
print(f"Salário do João: R$ {joao_salario}")

# Iterando sobre funcionários
print("\nFuncionários:")
for nome, dados in empresa["funcionarios"].items():
    print(f"    {nome.title():} {dados['cargo']} - R$ {dados['salario']}")
```

Verificações e Testes

```
dados = {"a": 1, "b": 2, "c": 3}

# Verificando existência de chaves
print(f"'a' existe: {'a' in dados}") # True
print(f"'d' existe: {'d' in dados}") # False

# Verificando existência de valores
print(f"Valor 2 existe: {2 in dados.values()}") # True
print(f"Valor 4 existe: {4 in dados.values()}") # False

# Verificando existência de itens
print(f"('a', 1) existe: {'a', 1} in dados.items()") # True

# Usando not in
print(f"'d' não existe: {'d' not in dados}") # True
```

Exemplo: Sistema de Login

```
# Sistema de usuários
usuarios = {
    "admin": {
        "senha": "123456",
        "nivel": "administrador",
        "ativo": True
    },
    "usuario1": {
        "senha": "senha123",
        "nivel": "usuario",
        "ativo": True
    }
}

def verificar_login(usuario, senha):
    # Verificando se usuário existe
    if usuario not in usuarios:
```

```
    return False, "Usuário não encontrado"

# Acessando dados do usuário
dados = usuarios[usuario]

# Verificando senha e status
if dados["senha"] == senha and dados["ativo"]:
    return True, dados["nivel"]
elif not dados["ativo"]:
    return False, "Usuário inativo"
else:
    return False, "Senha incorreta"

# Testando o sistema
resultado, info = verificar_login("admin", "123456")
print(f"Login admin: {'Sucesso' if resultado else 'Falha'} - {info}")
```

Dicas de Acesso

- **Use get():** Para acesso seguro com valor padrão
- **Verifique existência:** Use `in` antes de acessar
- **Iteração eficiente:** Use `items()` para pares chave-valor
- **Dados aninhados:** Acesse nível por nível



PROGRAMA DE
FORMAÇÃO ACELERADA



 **CEPEDI**

 **Sof**

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Adição e Modificação

Adicionando Elementos

```
# Criando dicionário vazio
contato = {}

# Adicionando elementos um por um
contato["nome"] = "João Silva"
contato["telefone"] = "(71) 99999-1111"
contato["email"] = "joao@email.com"
contato["cidade"] = "Salvador"

print("Contato:", contato)
# {'nome': 'João Silva', 'telefone': '(71) 99999-1111', 'email': 'joao@email.com', 'cidade': 'Salvador'}

# Adicionando elemento que já existe (sobrescreve)
contato["telefone"] = "(71) 99999-2222"
print("Telefone atualizado:", contato["telefone"])
```



Modificando Valores

```
# Dicionário inicial
produto = {
    "nome": "Notebook",
    "preco": 2500.00,
    "estoque": 10
}

print("Produto original:", produto)

# Modificando valores existentes
produto["preco"] = 2200.00 # Desconto
produto["estoque"] = 8     # Venda de 2 unidades

print("Produto modificado:", produto)

# Adicionando novos campos
produto["categoria"] = "Eletrônicos"
```



```
produto["garantia"] = "12 meses"

print("Produto com novos campos:", produto)
```

Método update()

```
# Dicionário base
pessoa = {
    "nome": "Maria",
    "idade": 25
}

# Atualizando com outro dicionário
novos_dados = {
    "profissao": "Engenheira",
    "cidade": "São Paulo",
    "telefone": "99999-3333"
}

pessoa.update(novos_dados)
print("Pessoa atualizada:", pessoa)

# Atualizando com pares chave-valor
pessoa.update(idade=26, salario=5000)
print("Pessoa com idade e salário:", pessoa)

# Atualizando com lista de tuplas
atualizacoes = [("departamento", "TI"), ("ativo", True)]
pessoa.update(atualizacoes)
print("Pessoa final:", pessoa)
```



Modificação Condicional

```
# Sistema de inventário
inventario = {
    "notebook": {"preco": 2500, "estoque": 5},
    "mouse": {"preco": 50, "estoque": 20},
    "teclado": {"preco": 120, "estoque": 15}
}

# Verificando e atualizando preços condicionalmente
produto = "notebook"
estoque_minimo = 10

# Verificando se produto existe
if produto in inventario:
    # Verificando se estoque está baixo
    if inventario[produto]["estoque"] < estoque_minimo:
        inventario[produto]["preco"] = 2000 # Desconto
        print(f"Preço do {produto} atualizado para R$ 2000")
    else:
        print(f"Estoque do {produto} está adequado")
```



```

else:
    print(f"Produto {produto} não encontrado")

# Verificando outro produto
produto = "mouse"
if produto in inventario:
    if inventario[produto]["estoque"] < estoque_minimo:
        inventario[produto]["preco"] = 45
        print(f"Preço do {produto} atualizado")
    else:
        print(f"Estoque do {produto} está adequado")

```



Modificação em Lote



```

# Sistema de notas
notas = {
    "Ana": [8.5, 7.0, 9.2],
    "Bruno": [6.8, 8.0, 7.5],
    "Carla": [9.0, 8.5, 9.5]
}

# Adicionando nova nota para todos os alunos
nova_nota = 8.0
for aluno in notas:
    notas[aluno].append(nova_nota)

print("Notas com nova prova:", notas)

# Calculando e adicionando médias
for aluno, lista_notas in notas.items():
    media = sum(lista_notas) / len(lista_notas)
    notas[aluno] = {
        "notas": lista_notas,
        "media": media,
        "status": "Aprovado" if media >= 7.0 else "Reprovado"
    }

print("Notas com médias:", notas)

```



Exemplo: Sistema de Vendas



```

# Sistema de vendas
vendas = {
    "janeiro": 15000,
    "fevereiro": 18000,
    "marco": 12000
}

# Adicionando nova venda
vendas["abril"] = 22000

```

```

# Atualizando venda existente (correção)
vendas["marco"] = 15000 # Correção de erro

# Adicionando múltiplas vendas
novas_vendas = {
    "maio": 19000,
    "junho": 21000
}
vendas.update(novas_vendas)

# Calculando total e média
total = sum(vendas.values())
media = total / len(vendas)

# Encontrando melhor mês (maior venda)
melhor_mes = "janeiro"
maior_venda = vendas["janeiro"]

for mes, valor in vendas.items():
    if valor > maior_venda:
        maior_venda = valor
        melhor_mes = mes

# Adicionando estatísticas
vendas["total"] = total
vendas["media"] = media
vendas["melhor_mes"] = melhor_mes

print("Vendas com estatísticas:", vendas)

```

⚠ Cuidados na Modificação

- **Chaves existentes:** Valores são sobrescritos
- **Chaves inexistentes:** Novos pares são criados
- **Performance:** update() é mais eficiente para múltiplas alterações
- **Backup:** Faça cópia antes de modificações importantes



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Remoção de Elementos

Métodos de Remoção

```
# Dicionário inicial
contato = {
    "nome": "João Silva",
    "telefone": "(71) 99999-1111",
    "email": "joao@email.com",
    "cidade": "Salvador",
    "idade": 25
}

print("Contato original:", contato)

# 1. Remoção com del
del contato["idade"]
print("Após remover 'idade':", contato)

# 2. Remoção com pop() - retorna o valor
telefone_removido = contato.pop("telefone")
print(f"Telefone removido: {telefone_removido}")
print("Após pop('telefone'):", contato)

# 3. Remoção com pop() e valor padrão
cidade_removida = contato.pop("cidade", "Não informado")
print(f"Cidade removida: {cidade_removida}")

# Tentativa de remover chave inexistente com valor padrão
estado_removido = contato.pop("estado", "Não informado")
print(f"Estado removido: {estado_removido}")
```



Método popitem()

popitem(): Remove e retorna o último item inserido no dicionário (LIFO - Last In, First Out)

```
# Dicionário para teste
dados = {
    "a": 1,
```



```
"b": 2,
"c": 3,
"d": 4
}

print("Dicionário original:", dados)

# Removendo o último item inserido (ordem de inserção em Python 3.7+)
item_removido = dados.popitem()
print(f"Item removido: {item_removido}")
print("Após popitem():", dados)

# Removendo mais itens
item_removido2 = dados.popitem()
print(f"Item removido: {item_removido2}")
print("Após segundo popitem():", dados)

# Demonstrando ordem LIFO (Last In, First Out)
print("\n--- Demonstração da ordem LIFO ---")
dados_teste = {"primeiro": 1, "segundo": 2, "terceiro": 3}
print("Dicionário:", dados_teste)

# Adicionando item por último
dados_teste["quarto"] = 4
print("Após adicionar 'quarto':", dados_teste)

# popitem() remove o último adicionado
item_removido = dados_teste.popitem()
print(f"Item removido (último adicionado): {item_removido}")
print("Dicionário após popitem():", dados_teste)

# Removendo todos os itens restantes
print("\n--- Removendo todos os itens ---")
while dados_teste:
    item = dados_teste.popitem()
    print(f"Removido: {item}")
print("Dicionário vazio:", dados_teste)
```

Limpeza Completa

```
# Dicionário para limpeza
inventario = {
    "produto1": {"preco": 100, "estoque": 10},
    "produto2": {"preco": 200, "estoque": 5},
    "produto3": {"preco": 300, "estoque": 8}
}

print("Inventário original:", inventario)

# Limpeza completa com clear()
inventario.clear()
print("Após clear():", inventario)
print(f"Tamanho após clear(): {len(inventario)}")

# Verificando se está vazio
print(f"Está vazio: {not inventario}")
```



```
print(f"Está vazio: {len(inventario) == 0}")
```

Remoção Condicional

```
# Sistema de usuários
usuarios = {
    "admin": {"ativo": True, "nivel": "administrador"},
    "user1": {"ativo": False, "nivel": "usuario"},
    "user2": {"ativo": True, "nivel": "usuario"},
    "user3": {"ativo": False, "nivel": "usuario"}
}

print("Usuários originais:", usuarios)

# Removendo usuários inativos
usuarios_ativos = {}
for usuario, dados in usuarios.items():
    if dados["ativo"]:
        usuarios_ativos[usuario] = dados
    else:
        print(f"Removendo usuário inativo: {usuario}")

usuarios = usuarios_ativos
print("Usuários ativos:", usuarios)

# Removendo usuários com nível específico
usuarios_finais = {}
for usuario, dados in usuarios.items():
    if dados["nivel"] != "administrador":
        usuarios_finais[usuario] = dados
    else:
        print(f"Mantendo administrador: {usuario}")

usuarios = usuarios_finais
print("Usuários finais:", usuarios)
```

Remoção em Lote

```
# Sistema de notas
notas = {
    "Ana": [8.5, 7.0, 9.2, 6.8],
    "Bruno": [6.8, 8.0, 7.5, 5.0],
    "Carla": [9.0, 8.5, 9.5, 8.0],
    "Diego": [7.2, 6.5, 8.0, 7.5]
}

print("Notas originais:", notas)

# Removendo alunos com média menor que 7.0
alunos_aprovados = {}
for aluno, lista_notas in notas.items():
    media = sum(lista_notas) / len(lista_notas)
```



```
if media >= 7.0:
    alunos_aprovados[aluno] = lista_notas
else:
    print(f"Removendo aluno reprovado: {aluno} (média: {media:.2f})")

notas = alunos_aprovados
print("Alunos aprovados:", notas)

# Removendo a menor nota de cada aluno
for aluno in notas:
    if len(notas[aluno]) > 1:
        # Encontrando a menor nota
        menor_nota = notas[aluno][0]
        for nota in notas[aluno]:
            if nota < menor_nota:
                menor_nota = nota

        notas[aluno].remove(menor_nota)
        print(f"Removida menor nota de {aluno}: {menor_nota}")

print("Notas após remoção da menor:", notas)
```

Exemplo: Sistema de Cache

```
# Sistema de cache com TTL (Time To Live)
cache = {
    "usuario_123": {"dados": "info", "timestamp": 1000},
    "usuario_456": {"dados": "info", "timestamp": 2000},
    "usuario_789": {"dados": "info", "timestamp": 3000}
}

print("Cache original:", cache)

# Simulando limpeza de cache expirado
tempo_atual = 2500
ttl = 1500 # Time To Live
itens_removidos = []

# Verificando cada item do cache
for chave, valor in list(cache.items()):
    if tempo_atual - valor["timestamp"] > ttl:
        del cache[chave]
        itens_removidos.append(chave)
        print(f"Item expirado removido: {chave}")

print(f"Itens removidos: {itens_removidos}")
print(f"Cache após limpeza: {cache}")

# Adicionando novo item ao cache
cache["usuario_999"] = {"dados": "nova_info", "timestamp": 2500}
print("Cache com novo item:", cache)
```



⚠ Cuidados na Remoção

- **del:** Gera erro se chave não existir
- **pop():** Gera erro se chave não existir (sem valor padrão)
- **popitem():** Gera erro em dicionário vazio
- **clear():** Remove todos os itens de uma vez



BOLSA
FUTURO
DIGITAL

PROGRAMA DE
FORMAÇÃO ACELERADA



INSTITUTO
FEDERAL
Bahia



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Métodos de Dicionários

Métodos Principais

```
# Dicionário para demonstração
pessoa = {
    "nome": "Ana Silva",
    "idade": 28,
    "profissao": "Designer",
    "cidade": "Salvador"
}

# 1. keys() - retorna as chaves
chaves = pessoa.keys()
print("Chaves:", list(chaves)) # ['nome', 'idade', 'profissao', 'cidade']

# 2. values() - retorna os valores
valores = pessoa.values()
print("Valores:", list(valores)) # ['Ana Silva', 28, 'Designer', 'Salvador']

# 3. items() - retorna pares chave-valor
itens = pessoa.items()
print("Items:", list(itens)) # [('nome', 'Ana Silva'), ('idade', 28), ...]

# 4. get() - acesso seguro
nome = pessoa.get("nome", "Não informado")
telefone = pessoa.get("telefone", "Não informado")
print(f"Nome: {nome}, Telefone: {telefone}")
```



Métodos de Informação

```
# Dicionário para teste
dados = {"a": 1, "b": 2, "c": 3, "d": 4}

# 1. len() - tamanho do dicionário
print(f"Tamanho: {len(dados)}") # 4

# 2. in / not in - verificação de existência
print(f"'a' existe: {'a' in dados}") # True
print(f"'e' existe: {'e' in dados}") # False
```



```
print(f"'e' não existe: {'e' not in dados}") # True

# 3. isinstance() - verificação de tipo
print(f"É dicionário: {isinstance(dados, dict)}") # True
print(f"É lista: {isinstance(dados, list)}") # False

# 4. copy() - cópia do dicionário
copia = dados.copy()
print(f"Cópia: {copia}")
print(f"São o mesmo objeto: {dados is copia}") # False
```



Métodos de Modificação

```
# Dicionário inicial
contato = {"nome": "João", "idade": 25}

# 1. update() - atualização em lote
novos_dados = {"telefone": "99999-1111", "email": "joao@email.com"}
contato.update(novos_dados)
print("Após update():", contato)

# 2. setdefault() - define valor se chave não existir
contato.setdefault("cidade", "Salvador")
contato.setdefault("nome", "João Silva") # Não altera se já existir
print("Após setdefault():", contato)

# 3. pop() - remove e retorna valor
idade = contato.pop("idade")
print(f"Idade removida: {idade}")
print("Após pop('idade'):", contato)

# 4. popitem() - remove e retorna item arbitrário
item = contato.popitem()
print(f"Item removido: {item}")
print("Após popitem():", contato)
```



Métodos de Busca

```
# Dicionário com dados repetidos
notas = {
    "Ana": [8.5, 7.0, 9.2],
    "Bruno": [6.8, 8.0, 7.5],
    "Carla": [9.0, 8.5, 9.5],
    "Diego": [7.2, 6.5, 8.0]
}

# 1. Buscando chave com valor máximo
melhor_aluno = max(notas, key=lambda x: sum(notas[x])/len(notas[x]))
print(f"Melhor aluno: {melhor_aluno}")

# 2. Buscando chave com valor mínimo
pior_aluno = min(notas, key=lambda x: sum(notas[x])/len(notas[x]))
```



```
print(f"Pior aluno: {pior_aluno}")

# 3. Verificando se todos os valores atendem condição
todos_aprovados = all(sum(notas[aluno])/len(notas[aluno]) >= 7.0 for aluno in notas)
print(f"Todos aprovados: {todos_aprovados}")

# 4. Verificando se algum valor atende condição
algum_aprovado = any(sum(notas[aluno])/len(notas[aluno]) >= 7.0 for aluno in notas)
print(f"Algum aprovado: {algum_aprovado}")
```



Métodos de Agregação

```
# Dicionário com valores numéricos
vendas = {
    "janeiro": 15000,
    "fevereiro": 18000,
    "marco": 12000,
    "abril": 22000,
    "maio": 19000
}

# 1. sum() - soma dos valores
total_vendas = sum(vendas.values())
print(f"Total de vendas: {total_vendas}")

# 2. max() - valor máximo
maior_venda = max(vendas.values())
print(f"Maior venda: {maior_venda}")

# 3. min() - valor mínimo
menor_venda = min(vendas.values())
print(f"Menor venda: {menor_venda}")

# 4. Média
media_vendas = total_vendas / len(vendas)
print(f"Média de vendas: {media_vendas:.2f}")

# 5. Mês com maior venda
mes_maior_venda = max(vendas, key=vendas.get)
print(f"Mês com maior venda: {mes_maior_venda}")
```



Exemplo: Sistema de Estatísticas

```
# Sistema de estatísticas de usuários
usuarios = {
    "admin": {"login": 100, "tempo_online": 500},
    "user1": {"login": 50, "tempo_online": 200},
    "user2": {"login": 75, "tempo_online": 300},
    "user3": {"login": 25, "tempo_online": 100}
}
```



```
# Calculando estatísticas
total_logins = sum(user["login"] for user in usuarios.values())
total_tempo = sum(user["tempo_online"] for user in usuarios.values())
media_logins = total_logins / len(usuarios)
media_tempo = total_tempo / len(usuarios)

# Usuário mais ativo
usuario_mais_ativo = max(usuarios, key=lambda x: usuarios[x]["login"])
tempo_mais_alto = max(usuarios, key=lambda x: usuarios[x]["tempo_online"])

print(f"Total de logins: {total_logins}")
print(f"Total de tempo online: {total_tempo}")
print(f"Média de logins: {media_logins:.2f}")
print(f"Média de tempo: {media_tempo:.2f}")
print(f"Usuário mais ativo: {usuario_mais_ativo}")
print(f"Usuário com mais tempo: {tempo_mais_alto}")
```

Resumo dos Métodos

Método	Função	Exemplo
keys()	Retorna as chaves	dicio.keys()
values()	Retorna os valores	dicio.values()
items()	Retorna pares chave-valor	dicio.items()
get()	Acesso seguro com valor padrão	dicio.get("chave", "padrão")



PROGRAMA DE
FORMAÇÃO ACELERADA



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Funções Built-in

Funções para Dicionários



```
# Dicionário para demonstração
vendas = {
    "janeiro": 15000,
    "fevereiro": 18000,
    "marco": 12000,
    "abril": 22000,
    "maio": 19000
}

# 1. len() - tamanho do dicionário
print(f"Quantidade de meses: {len(vendas)}") # 5

# 2. max() - valor máximo
maior_venda = max(vendas.values())
print(f"Maior venda: {maior_venda}") # 22000

# 3. min() - valor mínimo
menor_venda = min(vendas.values())
print(f"Menor venda: {menor_venda}") # 12000

# 4. sum() - soma dos valores
total_vendas = sum(vendas.values())
print(f"Total de vendas: {total_vendas}") # 86000

# 5. Média
media_vendas = total_vendas / len(vendas)
print(f"Média de vendas: {media_vendas:.2f}") # 17200.00
```

Funções de Busca

```
# Dicionário com dados complexos
alunos = {
    "Ana": {"notas": [8.5, 7.0, 9.2], "idade": 20},
    "Bruno": {"notas": [6.8, 8.0, 7.5], "idade": 19},
    "Carla": {"notas": [9.0, 8.5, 9.5], "idade": 21},
    "Diego": {"notas": [7.2, 6.5, 8.0], "idade": 20}
```

```
}

# 1. max() com função personalizada
melhor_aluno = max(alunos, key=lambda x: sum(alunos[x]["notas"])/len(alunos[x]["notas"]))
print(f"Melhor aluno: {melhor_aluno}")

# 2. min() com função personalizada
pior_aluno = min(alunos, key=lambda x: sum(alunos[x]["notas"])/len(alunos[x]["notas"]))
print(f"Pior aluno: {pior_aluno}")

# 3. sorted() - ordenação
alunos_ordenados = sorted(alunos, key=lambda x: sum(alunos[x]["notas"])/len(alunos[x]["notas"]))
print(f"Alunos ordenados por nota: {alunos_ordenados}")

# 4. filter() - filtragem
alunos_aprovados = list(filter(lambda x: sum(alunos[x]["notas"])/len(alunos[x]["notas"]) > 7.0, alunos))
print(f"Alunos aprovados: {alunos_aprovados}")
```



Funções de Agregação



```
# Dicionário com dados numéricos
produtos = {
    "notebook": {"preco": 2500, "estoque": 10},
    "mouse": {"preco": 50, "estoque": 25},
    "teclado": {"preco": 120, "estoque": 15},
    "monitor": {"preco": 800, "estoque": 8}
}

# 1. Soma de preços
total_precos = sum(produto["preco"] for produto in produtos.values())
print(f"Total de preços: R$ {total_precos}")

# 2. Soma de estoque
total_estoque = sum(produto["estoque"] for produto in produtos.values())
print(f"Total de estoque: {total_estoque} unidades")

# 3. Valor total do inventário
valor_inventario = sum(produto["preco"] * produto["estoque"] for produto in produtos.values())
print(f"Valor total do inventário: R$ {valor_inventario}")

# 4. Produto mais caro
produto_mais_caro = max(produtos, key=lambda x: produtos[x]["preco"])
print(f"Produto mais caro: {produto_mais_caro}")

# 5. Produto com maior estoque
produto_maior_estoque = max(produtos, key=lambda x: produtos[x]["estoque"])
print(f"Produto com maior estoque: {produto_maior_estoque}")
```



Funções de Transformação

```
# Dicionário original
```

```
dados = {
    "nome": "João Silva",
    "idade": 25,
    "salario": 5000,
    "departamento": "TI"
}

# 1. map() - transformação de valores
dados_upper = {k: v.upper() if isinstance(v, str) else v for k, v in dados.items()}
print("Dados em maiúsculo:", dados_upper)

# 2. filter() - filtragem de itens
dados_numericos = {k: v for k, v in dados.items() if isinstance(v, (int, float))}
print("Dados numéricos:", dados_numericos)

# 3. Transformação com função personalizada
def calcular_bonus(salario):
    return salario * 0.1

dados_com_bonus = {k: calcular_bonus(v) if k == "salario" else v for k, v in dados.items()}
print("Dados com bônus:", dados_com_bonus)

# 4. Inversão de chave-valor
dados_invertidos = {v: k for k, v in dados.items()}
print("Dados invertidos:", dados_invertidos)
```

Funções de Análise

```
# Dicionário com dados de vendas
vendas_mensais = {
    "janeiro": 15000,
    "fevereiro": 18000,
    "marco": 12000,
    "abril": 22000,
    "maio": 19000,
    "junho": 21000
}

# 1. Análise de tendência
valores = list(vendas_mensais.values())
crescimento = valores[-1] - valores[0]
print(f"Crescimento total: R$ {crescimento}")

# 2. Mês com maior crescimento
crescimento_mensal = {}
meses = list(vendas_mensais.keys())
for i in range(1, len(meses)):
    mes_atual = meses[i]
    mes_anterior = meses[i-1]
    crescimento_mensal[mes_atual] = vendas_mensais[mes_atual] - vendas_mensais[mes_anterior]

mes_maior_crescimento = max(crescimento_mensal, key=crescimento_mensal.get)
print(f"Mês com maior crescimento: {mes_maior_crescimento}")

# 3. Análise de variabilidade
media = sum(valores) / len(valores)
variancia = sum((x - media) ** 2 for x in valores) / len(valores)
```

```
desvio_padrao = variancia ** 0.5
print(f"Média: R$ {media:.2f}")
print(f"Desvio padrão: R$ {desvio_padrao:.2f}")
```

Exemplo: Sistema de Análise de Dados

```
# Sistema de análise de dados de funcionários
funcionarios = {
    "Ana": {"salario": 5000, "anos_experiencia": 3, "departamento": "TI"},
    "Bruno": {"salario": 4500, "anos_experiencia": 2, "departamento": "TI"},
    "Carla": {"salario": 6000, "anos_experiencia": 5, "departamento": "RH"},
    "Diego": {"salario": 4000, "anos_experiencia": 1, "departamento": "TI"},
    "Elena": {"salario": 5500, "anos_experiencia": 4, "departamento": "RH"}
}

# Análise por departamento
departamentos = {}
for nome, dados in funcionarios.items():
    dept = dados["departamento"]
    if dept not in departamentos:
        departamentos[dept] = {"funcionarios": [], "salarios": []}
    departamentos[dept]["funcionarios"].append(nome)
    departamentos[dept]["salarios"].append(dados["salario"])

# Estatísticas por departamento
for dept, info in departamentos.items():
    salarios = info["salarios"]
    print(f"\n{dept}:")
    print(f"  Funcionários: {len(info['funcionarios'])}")
    print(f"  Salário médio: R$ {sum(salarios)/len(salarios):.2f}")
    print(f"  Salário máximo: R$ {max(salarios)}")
    print(f"  Salário mínimo: R$ {min(salarios)}")
```

Dicas de Uso

- **max/min:** Use com `key=` para critérios personalizados
- **sum:** Funciona com valores numéricos
- **len:** Conta o número de pares chave-valor
- **sorted:** Retorna lista ordenada de chaves



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Exercícios - Tuplas

Exercício 1: Dados Pessoais

```
# Crie uma tupla com seus dados pessoais
# (nome, idade, profissão, cidade)
# Acesse cada elemento e exiba as informações

# Solução:
dados_pessoais = ("João Silva", 25, "Programador", "Salvador")

# Acessando elementos
nome = dados_pessoais[0]
idade = dados_pessoais[1]
profissao = dados_pessoais[2]
cidade = dados_pessoais[3]

print(f"Nome: {nome}")
print(f"Idade: {idade}")
print(f"Profissão: {profissao}")
print(f"Cidade: {cidade}")

# Desempacotamento
nome, idade, profissao, cidade = dados_pessoais
print(f"Desempacotamento: {nome} tem {idade} anos")
```



Exercício 2: Coordenadas Geográficas

```
# Crie tuplas representando coordenadas de cidades
# Calcule a distância entre duas cidades

# Solução:
salvador = (-12.9714, -38.5014)
sao_paulo = (-23.5505, -46.6333)

# Desempacotando coordenadas
lat1, lon1 = salvador
lat2, lon2 = sao_paulo

# Calculando distância (fórmula simplificada)
```



```
distancia = ((lat2 - lat1)**2 + (lon2 - lon1)**2)**0.5
print(f"Distância Salvador-SP: {distancia:.2f} graus")

# Criando tupla de resultado
resultado = (distancia, "graus")
print(f"Resultado: {resultado}")
```

Exercício 3: Sistema de Notas

```
# Crie uma tupla com notas de um aluno
# Calcule a média e determine se foi aprovado

# Solução:
notas = (8.5, 7.0, 9.2, 6.8, 8.0)

# Calculando média
soma = sum(notas)
media = soma / len(notas)

# Determinando status
status = "Aprovado" if media >= 7.0 else "Reprovado"

print(f"Notas: {notas}")
print(f"Média: {media:.2f}")
print(f"Status: {status}")

# Usando funções built-in
print(f"Maior nota: {max(notas)}")
print(f"Menor nota: {min(notas)}")
print(f"Total de notas: {len(notas)}")
```



Exercício 4: Análise de Dados

```
# Crie uma tupla com dados de vendas mensais
# Analise os dados e encontre o melhor mês

# Solução:
vendas = (15000, 18000, 12000, 22000, 19000)
meses = ("Janeiro", "Fevereiro", "Março", "Abril", "Maio")

# Encontrando melhor mês
maior_venda = max(vendas)
indice_melhor = vendas.index(maior_venda)
melhor_mes = meses[indice_melhor]

# Estatísticas
total_vendas = sum(vendas)
media_vendas = total_vendas / len(vendas)

print(f"Vendas: {vendas}")
print(f"Melhor mês: {melhor_mes} (R$ {maior_venda})")
print(f"Total: R$ {total_vendas}")
```



```
print(f"Média: R$ {media_vendas:.2f}")

# Usando zip para combinar dados
dados_combinados = list(zip(meses, vendas))
print(f"Dados combinados: {dados_combinados}")
```



Exercício 5: Sistema de Pontuação

```
# Crie um sistema de pontuação para um jogo
# Calcule estatísticas dos jogadores

# Solução:
pontuacoes = (100, 150, 200, 150, 300, 150, 250)

# Análise das pontuações
total_pontos = sum(pontuacoes)
pontuacao_maxima = max(pontuacoes)
pontuacao_minima = min(pontuacoes)
media_pontos = total_pontos / len(pontuacoes)

# Contando pontuações específicas
qtd_150 = pontuacoes.count(150)
pos_300 = pontuacoes.index(300)

print(f"Pontuações: {pontuacoes}")
print(f"Total: {total_pontos}")
print(f"Máxima: {pontuacao_maxima}")
print(f"Mínima: {pontuacao_minima}")
print(f"Média: {media_pontos:.2f}")
print(f"Quantas vezes 150: {qtd_150}")
print(f"Posição do 300: {pos_300}")

# Criando tupla de estatísticas
estatisticas = (total_pontos, pontuacao_maxima, pontuacao_minima, media_pontos)
print(f"Estatísticas: {estatisticas}")
```



Exercício 6: Tupla Aninhada

```
# Crie uma tupla contendo outras tuplas
# Represente pontos em um plano cartesiano

# Solução:
pontos = (
    (0, 0),      # Origem
    (3, 4),      # Ponto A
    (6, 8),      # Ponto B
    (9, 12),     # Ponto C
)

print("Pontos:", pontos)

# Calculando distâncias da origem
```

```
for i, ponto in enumerate(pontos):
    x, y = ponto
    distancia = (x**2 + y**2)**0.5
    print(f"Ponto {i}: {ponto} - Distância da origem: {distancia:.2f}")

# Encontrando ponto mais distante
distancias = []
for ponto in pontos:
    x, y = ponto
    distancia = (x**2 + y**2)**0.5
    distancias.append(distancia)

maior_distancia = max(distancias)
indice_maior = distancias.index(maior_distancia)
ponto_mais_distante = pontos[indice_maior]

print(f"Ponto mais distante: {ponto_mais_distante} (distância: {maior_distancia:.2f})")
```



Exemplo: Sistema de Coordenadas

```
# Sistema de coordenadas para um jogo
coordenadas_jogo = (
    (0, 0),      # Spawn
    (10, 5),     # Checkpoint 1
    (20, 10),    # Checkpoint 2
    (30, 15),    # Checkpoint 3
    (40, 20),    # Final
)

# Calculando distância total do percurso
distancia_total = 0
for i in range(1, len(coordenadas_jogo)):
    x1, y1 = coordenadas_jogo[i-1]
    x2, y2 = coordenadas_jogo[i]
    distancia = ((x2 - x1)**2 + (y2 - y1)**2)**0.5
    distancia_total += distancia

print(f"Distância total do percurso: {distancia_total:.2f} unidades")
```



Dicas para os Exercícios

- **Desempacotamento:** Use para acessar elementos facilmente
- **Funções built-in:** Aproveite max(), min(), sum(), len()
- **Métodos:** Use count() e index() quando necessário
- **Fatiamento:** Extraia partes da tupla quando útil



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Exercícios - Dicionários

Exercício 1: Agenda Telefônica



```
# Crie uma agenda telefônica usando dicionários
# Adicione, busque e atualize contatos

# Solução:
agenda = {}

# Adicionando contatos
agenda["João"] = {"telefone": "99999-1111", "email": "joao@email.com"}
agenda["Maria"] = {"telefone": "99999-2222", "email": "maria@email.com"}
agenda["Pedro"] = {"telefone": "99999-3333", "email": "pedro@email.com"}

print("Agenda:", agenda)

# Buscando contato
nome_busca = "Maria"
if nome_busca in agenda:
    contato = agenda[nome_busca]
    print(f"{nome_busca}: {contato['telefone']}")
else:
    print(f"Contato {nome_busca} não encontrado")

# Atualizando contato
agenda["João"]["telefone"] = "99999-1122"
print(f"Telefone do João atualizado: {agenda['João']['telefone']}")

# Listando todos os contatos
for nome, dados in agenda.items():
    print(f"{nome}: {dados['telefone']} - {dados['email']}")
```

Exercício 2: Sistema de Notas

```
# Crie um sistema de notas usando dicionários
# Calcule médias e determine status dos alunos

# Solução:
notas_alunos = {
```

```
"Ana": [8.5, 7.0, 9.2],
"Bruno": [6.8, 8.0, 7.5],
"Carla": [9.0, 8.5, 9.5],
"Diego": [7.2, 6.5, 8.0]
}

print("Sistema de Notas:")
for aluno, notas in notas_alunos.items():
    media = sum(notas) / len(notas)
    status = "Aprovado" if media >= 7.0 else "Reprovado"

    print(f"{aluno}:")
    print(f"  Notas: {notas}")
    print(f"  Média: {media:.2f}")
    print(f"  Status: {status}")

# Encontrando melhor aluno
melhor_aluno = max(notas_alunos, key=lambda x: sum(notas_alunos[x])/len(notas_alunos[x]))
print(f"Melhor aluno: {melhor_aluno}")

# Adicionando nova nota
notas_alunos["Ana"].append(8.8)
print(f"Ana com nova nota: {notas_alunos['Ana']}")
```

Exercício 3: Inventário de Produtos



```
# Crie um sistema de inventário com dicionários
# Gerencie produtos, preços e estoque

# Solução:
inventario = {
    "notebook": {"preco": 2500.00, "estoque": 10, "categoria": "Eletrônicos"},
    "mouse": {"preco": 50.00, "estoque": 25, "categoria": "Acessórios"},
    "teclado": {"preco": 120.00, "estoque": 15, "categoria": "Acessórios"}
}

# Calculando valor total do inventário
valor_total = 0
for produto, dados in inventario.items():
    valor_produto = dados["preco"] * dados["estoque"]
    valor_total += valor_produto

    print(f"{produto.title()}:")
    print(f"  Preço: R$ {dados['preco']:.2f}")
    print(f"  Estoque: {dados['estoque']} unidades")
    print(f"  Valor total: R$ {valor_produto:.2f}")

print(f"Valor total do inventário: R$ {valor_total:.2f}")

# Produtos por categoria
categorias = {}
for produto, dados in inventario.items():
    categoria = dados["categoria"]
    if categoria not in categorias:
        categorias[categoria] = []
    categorias[categoria].append(produto)
```

```
print("Produtos por categoria:", categorias)
```



Exercício 4: Sistema de Vendas



```
# Crie um sistema de vendas mensais
# Analise dados e gere relatórios

# Solução:
vendas = {
    "janeiro": 15000,
    "fevereiro": 18000,
    "marco": 12000,
    "abril": 22000,
    "maio": 19000,
    "junho": 21000
}

# Estatísticas básicas
total_vendas = sum(vendas.values())
media_vendas = total_vendas / len(vendas)
maior_venda = max(vendas.values())
menor_venda = min(vendas.values())

print(f"Total de vendas: R$ {total_vendas}")
print(f"Média mensal: R$ {media_vendas:.2f}")
print(f"Maior venda: R$ {maior_venda}")
print(f"Menor venda: R$ {menor_venda}")

# Mês com maior venda
mes_maior_venda = max(vendas, key=vendas.get)
print(f"Mês com maior venda: {mes_maior_venda} (R$ {vendas[mes_maior_venda]})")

# Crescimento mensal
meses = list(vendas.keys())
for i in range(1, len(meses)):
    mes_atual = meses[i]
    mes_anterior = meses[i-1]
    crescimento = vendas[mes_atual] - vendas[mes_anterior]
    print(f"Crescimento de {mes_anterior} para {mes_atual}: R$ {crescimento}")
```



Exercício 5: Sistema de Login



```
# Crie um sistema de login com usuários e senhas
# Implemente autenticação e níveis de acesso

# Solução:
usuarios = {
    "admin": {"senha": "123456", "nivel": "administrador", "ativo": True},
    "usuario1": {"senha": "senha123", "nivel": "usuario", "ativo": True},
    "usuario2": {"senha": "minhasenha", "nivel": "usuario", "ativo": False}
}
```

```
def verificar_login(usuario, senha):
    if usuario not in usuarios:
        return False, "Usuário não encontrado"

    dados = usuarios[usuario]
    if dados["senha"] == senha and dados["ativo"]:
        return True, dados["nivel"]
    elif not dados["ativo"]:
        return False, "Usuário inativo"
    else:
        return False, "Senha incorreta"

# Testando o sistema
testes = [
    ("admin", "123456"),
    ("usuario1", "senha_errada"),
    ("usuario2", "minhasenha"),
    ("inexistente", "qualquer")
]

for usuario, senha in testes:
    resultado, info = verificar_login(usuario, senha)
    status = "Sucesso" if resultado else "Falha"
    print(f>Login '{usuario}': {status} - {info}")
```



Exemplo: Sistema de Configuração



```
# Sistema de configuração de aplicação
config = {
    "database": {
        "host": "localhost",
        "port": 5432,
        "name": "meuapp"
    },
    "server": {
        "host": "0.0.0.0",
        "port": 8080,
        "debug": True
    },
    "logging": {
        "level": "INFO",
        "file": "app.log"
    }
}

# Acessando configurações
db_config = config["database"]
print(f"Banco: {db_config['host']}:{db_config['port']}")
print(f"Debug: {config['server']['debug']}")

# Atualizando configurações
config["server"]["debug"] = False
config["database"]["port"] = 3306

print("Configuração atualizada:")
for secao, configuracoes in config.items():
```

```
print(f"{secao}:")  
for chave, valor in configuracoes.items():  
    print(f"  {chave}: {valor}")
```

⚠ Dicas para os Exercícios

- **Acesso seguro:** Use `get()` para evitar erros
- **Verificação:** Use `in` antes de acessar
- **Iteração:** Use `items()` para chave-valor
- **Funções:** Aproveite `max()`, `min()` com `key=`



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Exercícios Combinados

Combinando Tuplas e Dicionários

```
# Sistema de coordenadas de cidades com informações adicionais
cidades = {
    "Salvador": {
        "coordenadas": (-12.9714, -38.5014),
        "populacao": 2886698,
        "estado": "BA"
    },
    "São Paulo": {
        "coordenadas": (-23.5505, -46.6333),
        "populacao": 12396372,
        "estado": "SP"
    },
    "Rio de Janeiro": {
        "coordenadas": (-22.9068, -43.1729),
        "populacao": 6775561,
        "estado": "RJ"
    }
}

# Exibindo informações das cidades
for cidade, dados in cidades.items():
    lat, lon = dados["coordenadas"] # Desempacotamento da tupla
    print(f"{cidade} ({dados['estado']}):")
    print(f"  Coordenadas: {lat}, {lon}")
    print(f"  População: {dados['populacao']:,}")

# Calculando distância entre Salvador e São Paulo
salvador = cidades["Salvador"]["coordenadas"]
sp = cidades["São Paulo"]["coordenadas"]
distancia = ((salvador[0] - sp[0])**2 + (salvador[1] - sp[1])**2)**0.5
print(f"Distância Salvador-SP: {distancia:.2f} graus")
```



Sistema de Competição

```
# Sistema de competição esportiva
competicao = {
```



```
"equipe_a": {
    "jogadores": ("João", "Maria", "Pedro", "Ana"),
    "pontuacoes": (100, 150, 200, 180),
    "categoria": "Profissional"
},
"equipe_b": {
    "jogadores": ("Bruno", "Carla", "Diego", "Elena"),
    "pontuacoes": (120, 140, 190, 160),
    "categoria": "Profissional"
},
"equipe_c": {
    "jogadores": ("Felipe", "Gabi", "Hugo", "Isa"),
    "pontuacoes": (90, 110, 130, 140),
    "categoria": "Amador"
}
}

# Analisando resultados
for equipe, dados in competicao.items():
    jogadores = dados["jogadores"]
    pontuacoes = dados["pontuacoes"]
    categoria = dados["categoria"]

    total_pontos = sum(pontuacoes)
    media_pontos = total_pontos / len(pontuacoes)

    print(f"\n{equipe.upper()} ({categoria}):")
    print(f"  Jogadores: {' '.join(jogadores)}")
    print(f"  Pontuações: {pontuacoes}")
    print(f"  Total: {total_pontos}")
    print(f"  Média: {media_pontos:.2f}")

# Melhor jogador da equipe
melhor_pontuacao = max(pontuacoes)
indice_melhor = pontuacoes.index(melhor_pontuacao)
melhor_jogador = jogadores[indice_melhor]
print(f"  Melhor jogador: {melhor_jogador} ({melhor_pontuacao} pontos)")

# Equipe vencedora
equipe_vencedora = max(competicao, key=lambda x: sum(competicao[x]["pontuacoes"]))
print(f"\nEquipe vencedora: {equipe_vencedora.upper()}")
```



Sistema de Análise de Dados

```
# Sistema de análise de vendas por região
vendas_regionais = {
    "norte": {
        "cidades": ("Manaus", "Belém", "Boa Vista"),
        "vendas_mensais": (50000, 45000, 55000, 48000),
        "meta_anual": 200000
    },
    "nordeste": {
        "cidades": ("Salvador", "Recife", "Fortaleza"),
        "vendas_mensais": (80000, 75000, 85000, 78000),
        "meta_anual": 320000
    },
    "sudeste": {
```

```
"cidades": ("São Paulo", "Rio de Janeiro", "Belo Horizonte"),
"vendas_mensais": (120000, 115000, 125000, 118000),
"meta_anual": 480000
}

}

# Análise por região
total_geral = 0
for regioao, dados in vendas_regionais.items():
    cidades = dados["cidades"]
    vendas = dados["vendas_mensais"]
    meta = dados["meta_anual"]

    total_vendas = sum(vendas)
    total_geral += total_vendas
    media_mensal = total_vendas / len(vendas)
    percentual_meta = (total_vendas / meta) * 100

    print(f"\n{regiao.upper()}:")
    print(f"  Cidades: {'', '.join(cidades)}")
    print(f"  Vendas mensais: {vendas}")
    print(f"  Total vendido: R$ {total_vendas:,}")
    print(f"  Média mensal: R$ {media_mensal:,.2f}")
    print(f"  Meta anual: R$ {meta:,}")
    print(f"  Atingimento: {percentual_meta:.1f}%")

print(f"\nTotal geral: R$ {total_geral:,}")

# Melhor região
melhor_regiao = max(vendas_regionais, key=lambda x: sum(vendas_regionais[x]["vendas_mensa
print(f"Melhor região: {melhor_regiao.upper()}")
```

Sistema Acadêmico



```
# Sistema acadêmico com múltiplas disciplinas
alunos = {
    "Ana Silva": {
        "matricula": "2024001",
        "curso": "Ciência da Computação",
        "disciplinas": {
            "Programação": (8.5, 7.0, 9.2),
            "Matemática": (7.5, 8.0, 8.5),
            "Física": (6.8, 7.2, 8.0)
        }
    },
    "Bruno Costa": {
        "matricula": "2024002",
        "curso": "Engenharia",
        "disciplinas": {
            "Programação": (7.0, 8.0, 7.5),
            "Matemática": (8.5, 9.0, 8.8),
            "Física": (9.0, 8.5, 9.2)
        }
    }
}

# Análise acadêmica
```

```
for aluno, dados in alunos.items():
    print(f"\n{aluno} ({dados['matricula']}) - {dados['curso']}")

    todas_notas = []
    for disciplina, notas in dados["disciplinas"].items():
        media_disciplina = sum(notas) / len(notas)
        todas_notas.extend(notas)
        status = "Aprovado" if media_disciplina >= 7.0 else "Reprovado"

    print(f"  {disciplina}:")
    print(f"    Notas: {notas}")
    print(f"    Média: {media_disciplina:.2f} - {status}")

# Média geral do aluno
media_geral = sum(todas_notas) / len(todas_notas)
print(f"  Média Geral: {media_geral:.2f}")

# Estatísticas gerais
todas_medias = []
for aluno, dados in alunos.items():
    todas_notas = []
    for notas in dados["disciplinas"].values():
        todas_notas.extend(notas)
    media_aluno = sum(todas_notas) / len(todas_notas)
    todas_medias.append(media_aluno)

media_turma = sum(todas_medias) / len(todas_medias)
print(f"\nMédia da turma: {media_turma:.2f}")
```



Exemplo: Sistema de E-commerce

```
# Sistema de e-commerce com produtos e avaliações
produtos = {
    "notebook_gamer": {
        "categoria": "Eletrônicos",
        "preco": 3500.00,
        "especificacoes": ("Intel i7", "16GB RAM", "RTX 3060"),
        "avaliacoes": (4.5, 4.8, 4.2, 4.7, 4.6),
        "vendas_mensais": (25, 30, 28, 35)
    },
    "smartphone_premium": {
        "categoria": "Eletrônicos",
        "preco": 2200.00,
        "especificacoes": ("128GB", "Câmera 64MP", "5G"),
        "avaliacoes": (4.3, 4.6, 4.4, 4.5, 4.2),
        "vendas_mensais": (50, 45, 55, 48)
    }
}

# Relatório de produtos
for produto, dados in produtos.items():
    print(f"\n{produto.replace('_', ' ').title()}")
    print(f"  Preço: R$ {dados['preco']:.2f}")
    print(f"  Especificações: {'', '.join(dados['especificacoes'])}")

# Análise de avaliações
```

```
avaliacoes = dados['avaliacoes']
media_avaliacoes = sum(avaliacoes) / len(avaliacoes)
print(f" Avaliação média: {media_avaliacoes:.2f} ({len(avaliacoes)} avaliações)

# Análise de vendas
vendas = dados['vendas_mensais']
total_vendas = sum(vendas)
receita = total_vendas * dados['preco']
print(f" Vendas: {total_vendas} unidades")
print(f" Receita: R$ {receita:,.2f}")
```

💡 Vantagens da Combinação

- **Flexibilidade:** Tuplas para dados fixos, dicionários para flexíveis
- **Organização:** Estruturas hierárquicas bem definidas
- **Performance:** Acesso eficiente a dados estruturados
- **Manutenibilidade:** Código mais limpo e legível



BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Resumo e Próximos Passos

O que Aprendemos

Tuplas

-  Estrutura imutável e ordenada
-  Sintaxe com parênteses ()
-  Métodos count() e index()
-  Desempacotamento de valores
-  Performance superior às listas
-  Uso como chaves de dicionários

Dicionários

-  Estrutura chave-valor mutável
-  Sintaxe com chaves {}
-  Métodos get(), keys(), values(), items()
-  Operações CRUD completas
-  Acesso eficiente por chave
-  Estruturas aninhadas

Quando Usar Cada Estrutura

Situação	Use Tupla	Use Dicionário	Use Lista
Dados fixos	 Coordenadas, configurações		
Mapeamento		 Agenda, inventário	
Sequência dinâmica			 Lista de compras
Performance crítica	 Dados imutáveis	 Busca por chave	

Melhores Práticas

★ Tuplas

- **Use para dados que não mudam:** coordenadas, configurações, constantes
- **Desempacotamento:** aproveite para extrair valores facilmente
- **Retorno de funções:** retorne múltiplos valores como tupla
- **Chaves de dicionário:** use quando precisar de chaves compostas

★ Dicionários

- **Use get():** sempre prefira para acesso seguro
- **Verifique existência:** use `in` antes de acessar
- **Iteração eficiente:** use `items()` para pares chave-valor
- **Estruturas aninhadas:** organize dados hierárquicos

Próximos Tópicos

Temas para Estudar

- **Conjuntos (Sets):** Coleções de elementos únicos
- **List/Dict Comprehensions:** Criação eficiente de estruturas
- **Funções com *args e **kwargs:** Parâmetros dinâmicos
- **JSON e APIs:** Manipulação de dados web
- **Pandas DataFrames:** Análise de dados avançada
- **Algoritmos e estruturas:** Ordenação, busca, grafos

Aplicações Práticas

```
# Exemplo de aplicação real: Sistema de análise de logs
logs_sistema = [
    {"timestamp": "2025-01-15 10:30:00", "nivel": "INFO", "usuario": "admin"},
    {"timestamp": "2025-01-15 10:31:00", "nivel": "ERROR", "usuario": "user1"},
    {"timestamp": "2025-01-15 10:32:00", "nivel": "WARNING", "usuario": "user2"},
    {"timestamp": "2025-01-15 10:33:00", "nivel": "INFO", "usuario": "admin"}
]

# Análise com dicionários e tuplas
estatisticas = {}
for log in logs_sistema:
    nivel = log["nivel"]
    usuario = log["usuario"]

    # Usando tupla como chave composta
    chave = (nivel, usuario)

    if chave in estatisticas:
        estatisticas[chave] += 1
    else:
        estatisticas[chave] = 1

print("Estatísticas de logs (nivel, usuário): quantidade")
for (nivel, usuario), quantidade in estatisticas.items():
    print(f"  ({nivel}, {usuario}): {quantidade}")
```



Recursos para Continuar

Links Úteis

- **Documentação Python:** docs.python.org
- **Python Tutor:** pythontutor.com (visualizar execução)
- **Real Python:** realpython.com (tutoriais avançados)
- **LeetCode:** leetcode.com (prática de algoritmos)

- **GitHub:** github.com (projetos open source)

Conclusão

Parabéns!

Você dominou duas estruturas de dados fundamentais em Python! 🎉

Tuplas e dicionários são ferramentas poderosas que você usará constantemente no desenvolvimento de aplicações. Continue praticando e explorando suas possibilidades!

Próximo passo: Aplicar estes conceitos em projetos reais e explorar estruturas de dados mais avançadas.

 Continue Praticando! 

"A prática leva à perfeição!"

